`w2filter`: a certified exhaustive streaming filter for graphs with prescribed cycle spectrum

Antonio Guastella
antonioguastella99@gmail.com

July 25, 2026

Abstract

We describe `w2filter`, a small ($\approx$ 420 lines of C) exhaustive streaming filter for the `graph6` output of McKay's `nauty/geng`. Given a prescribed set W of allowed cycle lengths and a set P of allowed path lengths, the filter selects the graphs whose cycle spectrum is contained in W, checks 2-degeneracy and then classifies the *graftable pairs*: pairs (u, v) such that *every* simple u–v path has length in P. Every pruning step carries an explicit, human-checked correctness proof shipped with the source, including proven bounds for all DFS stacks; correctness is further supported by an independent reference implementation, fixed anchor classes with known census values and sanitizer-clean builds. On a single core of a commodity desktop the filter alone sustains $\approx 2\times10^6$ graphs/second, so the full pipeline runs at the speed of the `geng` generator, which made an exhaustive sweep of $\approx 10^{11}$ graphs feasible. The tool was developed for a computational campaign on a density threshold ($\rho_2 = 11/7$) arising in work on the Erdős–Gyárfás conjecture, but is parametric and reusable for any cycle-spectrum–constrained enumeration.

1 Motivation

A recurring task in extremal graph theory is the exhaustive enumeration of graphs whose *cycle spectrum*, the set of lengths of simple cycles, avoids prescribed values. Our own motivation comes from a campaign on the Erdős–Gyárfás conjecture (every graph of minimum degree 3 contains a cycle of length a power of 2), where minimal counterexamples to a density bound $\rho_2 = 11/7$ for 2-degenerate graphs decompose into *window-2 blocks*: 2-connected graphs whose cycles all have length in $W = \{3, 5, 6, 7\}$. The campaign required deciding whether the class of window-2 blocks with $(V, E) = (19, 28)$ contains a block admitting a *graftable pair*, which is a pair (u, v) such that every simple u–v path has length in $P = \{1, 3, 4, 5\}$; such a block would yield, by a one-step surgery ("graft"), an explicit counterexample to the density bound. The class holds on the order of 10^{11} graphs; our first filter, written with a general-purpose graph library, processed on the order of 10^3 graphs per second per core on the campaign machine, which put the sweep out of reach. `w2filter` removes the filter from the critical path: it sustains $\approx 2 \times 10^6$ graphs per second per core on pre-generated input, so the pipeline runs at the speed of the generator ($\approx 7.6 \times 10^4$ graphs per second per core end to end), while keeping every optimisation *certified*: each pruning step is accompanied by a proof that it never discards a graph incorrectly.

2 Problem statement

The filter reads `graph6` lines from standard input and, for parameters $W \subseteq \{3, \dots, 31\}$ (allowed cycle lengths, default $\{3, 5, 6, 7\}$) and $P \subseteq \{1, \dots, 31\}$ (allowed path lengths, default $\{1, 3, 4, 5\}$), reports every input graph G such that

1. every simple cycle of G has length in W;
2. G is 2-degenerate;

together with the list of its graftable pairs $\{(u, v) : \emptyset \neq T(u, v) \subseteq P\}$, where $T(u, v)$ denotes the set of lengths of *all* simple u–v paths.

3 Pipeline and correctness

The filter is a four-stage pipeline; the first stage is a fast *sufficient* test, the remaining stages are exact.

3.1 Fast kill: the back-edge test

A DFS from an arbitrary root assigns depths; if a non-tree edge (v, w) satisfies $\mathrm{depth}(v) - \mathrm{depth}(w) \geq L_{\max} := \max W$, the graph is rejected.

Lemma 1 (Soundness of the back-edge test)**.** *If the test fires, G contains a simple cycle of length $> L_{\max}$.*

Proof. Let a be the lowest common ancestor of v and w in the DFS tree. The two tree paths $v \to a$ and $w \to a$ meet only in a, so $v \to a \to w \to v$ (closing with the edge) is a simple cycle of length $(\mathrm{depth}\, v - \mathrm{depth}\, a) + (\mathrm{depth}\, w - \mathrm{depth}\, a) + 1 \geq (\mathrm{depth}\, v - \mathrm{depth}\, w) + 1 > L_{\max}$, using $\mathrm{depth}(a) \leq \mathrm{depth}(w)$. □

The argument only uses the LCA, so it is valid for the iterative DFS implementation, where w need not be an ancestor of v. The test is *not* necessary: survivors proceed to the exact stage. In the $(V, E) = (14, 20)$ benchmark it eliminates 87.3% of the input.

3.2 Exact stage: canonical cycle enumeration

Lemma 2 (Canonical enumeration)**.** *Every simple cycle C contains a unique edge of minimum index $s = \{a, b\}$, and corresponds bijectively to a simple path $b \to a$ using only edges of index $> s$, closed by s. Enumerating, for each s, all such paths (simplicity enforced by a vertex bitmask) therefore visits every simple cycle exactly once.*

Each closure reports its exact length, which is compared against W; the stage aborts at the first violation. If it completes, the cycle spectrum of G is certified to lie in W. (When the generator already guarantees part of the constraint, e.g. `geng -f` for C_4-freeness, the stage simply never sees the corresponding violations; correctness does not depend on the pre-filter.)

3.3 Degeneracy

2-degeneracy is decided by iterated removal of vertices of degree ≤ 2. Removability is monotone under removals, so the peeling order is immaterial (confluence) and "the peeling empties G" is equivalent to 2-degeneracy.

3.4 Graftable pairs with reachability pruning

For the surviving graphs (a cold path: in our campaign, a handful out of 10^{11}) the filter enumerates, for each pair (u, v), simple paths by DFS. Two mechanisms keep this exact: any closure of length $\notin P$ aborts the pair immediately; and a simple prefix of $P_{\max} := \max P$ edges that has not reached v can only produce closures of length $> P_{\max}$, all bad, as their existence is equivalent to reachability of v from the prefix end in the residual graph, which a bitmask BFS decides exactly. Thus the search never explores deeper than $P_{\max}$, yet answers the universal question $T(u, v) \subseteq P$ exactly.

3.5 Stack bounds

All DFS stacks are static arrays; their bounds are proven and guarded by assertions that remain active in optimised builds.

Lemma 3 (Grouped-siblings bound)**.** *In an iterative DFS over* simple paths *that pushes all children of a popped node and pops in LIFO order, the stack is at all times partitioned into groups, one per level of the active branch, each group being the not-yet-popped children of the node one level below. Every non-top group has already lost at least one element (the one whose pop created the group above), whence* $\text{top} \leq (n-1)(\Delta-1) + \Delta = n(\Delta-1) + 1$*. For* $n \leq 31$*:* $\text{top} \leq 900$*.*

Remark 4. *The vertex-marking DFS of the back-edge test does* not *satisfy the grouped-siblings bound because a vertex may be pushed once per incoming edge before its first pop; the correct bound is the push count* $2E + 1$*. The distinction is not academic: the original buffer sized by intuition (*64*) was sufficient only for* $E \leq 31$*, a latent overflow for dense inputs found precisely while writing this proof during review.*

4 Validation methodology

1. **Anchor classes.** Three anchor classes are shipped as `make test`: $(11, 15)$: 8 blocks of which 3 with graftable pairs (with the exact pair lists); $(12, 17)$ and $(14, 20)$: 2 saturated blocks each. The census values were first established by earlier campaign runs with an independent library-based implementation (July 2026) and have since been reproduced by three further implementations: the C filter, the `stdlib` port and the `networkx` checker of item 2, including the exact graftable-pair lists. The test target fails on any deviation in counts or pair lists.

2. **Independent verification.** `make verify` compares normalised outputs (blocks *and* graftable-pair lists) against `tests/indep_check.py`, an independent checker built on `networkx`; different `graph6` parser, cycle enumeration via `simple_cycles`, degeneracy via core numbers, path enumeration via `all_simple_paths`: no algorithm is shared with the C code. `make verify-param` repeats the comparison on a non-default window; `make verify-random` repeats it on random windows, vertex and edge counts drawn from a printed seed, so that any failure is reproducible. Both run in CI, which therefore explores a different slice of the parameter space at every push. A fast same-design `stdlib` port (`reference_filter.py`) is also shipped for environments without `networkx`.

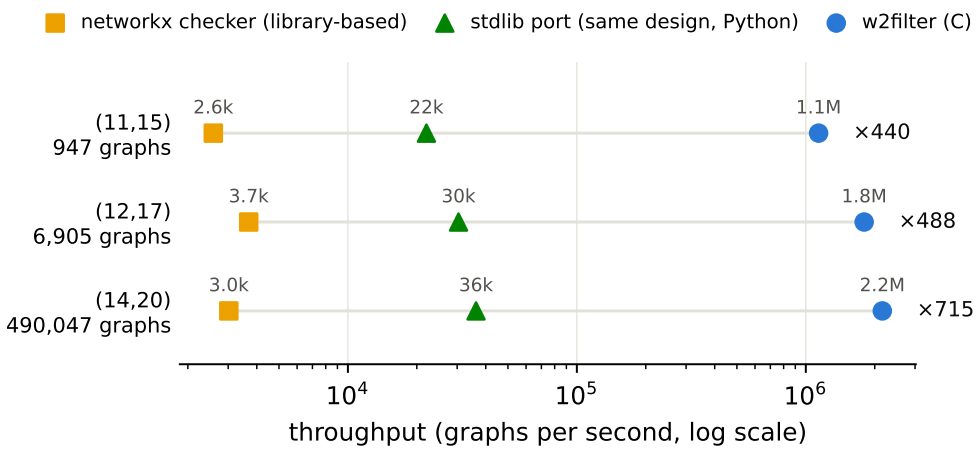

Figure 1: Throughput of the three shipped implementations on the anchor classes: single core, generation excluded, interpreter startup subtracted (`tests/bench_impls.py`, data shipped as `paper/impl_bench.csv`). The `networkx` checker stands in for the library-based design the campaign started from; the ratio at the right of each row is the C filter versus the checker.

3. **Sanitizers and CI.** Builds with `-fsanitize=address,undefined` run clean on the anchors; the CI matrix covers `gcc` and `clang` with `-Werror`.

4. **Domain guards.** Inputs outside $3 \le n \le 31$ are rejected explicitly (bitmask width), rather than producing undefined behaviour.

5 Benchmark

Single core (gcc `-O3`, commodity desktop), class $(V, E) = (14, 20)$, 490 047 graphs: on pre-generated input the filter sustains $\approx 2.2 \times 10^6$ graphs/s; end to end, including generation, the pipeline takes 6.45 s ($\approx 7.6 \times 10^4$ graphs/s), so the `geng` generator accounts for nearly all of the wall time. Stage statistics: back-edge kill 87.3%, exact stage 12.7%, degeneracy 0%, survivors 2. Figure 1 compares the three shipped implementations on the anchor classes: the C filter runs 50–60× faster than the `stdlib` port of the same design and 440–715× faster than the `networkx` checker, which stands in for the library-based approach the campaign started from. Absolute throughputs are specific to this machine and compiler, and the ratios also depend on the Python environment: in an independent re-run on different hardware (a single-vCPU cloud instance) the C throughput reproduced within 15% while the ratios compressed to ≈ 42–48× and ≈ 365–484×. What transfers is the order of magnitude, tens× over the port and hundreds× over the checker, together with the stage percentages, which are exact. Across 500 seeded random windows (`tests/window_stats.sh`, data in `paper/window_stats.csv`) the prefilter's kill share falls as $L_{\max}$ approaches the largest possible depth gap $n - 1$ and vanishes past it (52% of the sampled windows); the exact stage absorbs the load in that regime, with no effect on correctness. In production (8 workers via `geng` residue splitting, class $(19, 28)$, $\approx 10^{11}$ graphs) the generator, not the filter, is the bottleneck.

6 Limitations and reuse

Vertex counts are limited to $n \leq 31$ (32-bit masks); the source documents exactly which constants to enlarge for a 64-bit port, including the new stack bounds. The window and pair-set are parametric (`-cycles`, `-pairset`); users replacing the default window should check whether generator-side pre-filters (such as `geng -f`) remain valid for their parameters, or drop them: the exact stage re-checks every cycle length in either case. Throughout this note "certified" means that every pruning step is covered by a human-checked proof shipped with the source (`docs/proofs.md`); the proofs are not machine-checked, and formalising them is a natural further hardening step. Applications beyond our campaign include cycle-spectrum–constrained extremal problems, girth-type searches and counterexample hunts where certified exhaustiveness matters.

Availability

Source, proofs (`docs/proofs.md`), validation targets and CI configuration are distributed with this note. External dependency: `nauty` (B. D. McKay) for graph generation.

References

[1] B. D. McKay and A. Piperno, *Practical graph isomorphism, II*, J. Symbolic Comput. 60 (2014), 94–112.

[2] P. Erdős, *Some old and new problems in various branches of combinatorics*, Discrete Math. 165/166 (1997), 227–231.